

Project Deliverable: Network Monitoring & Alert System

Bikram Timalisina, Karan Bhatt, Rojan Tamang, and Sushant Ale Magar

Westcliff University

PRG 200: Introduction to Python

Professor Pandey

May 31, 2026

Project Deliverable: Network Monitoring & Alert System

Project Scope and Description

The goal of this project is to create a Network Monitoring & Alert system built in Python that constantly monitor network hosts and services, log performance data and alert the team to some anomaly that is detected which would trigger either a failure or an anomaly that would breach or cross a threshold.

What it does?

- Measure the round-trip time and packet loss and ping hosts (ICMP)
- Verify the operation of services (such as HTTP, DNS, SSH etc.) using a check of the TCP/UDP port(s)
- Gather metrics to persist in a local SQLite database from which to retrieve for historical monitoring
- Set email notifications when host(s) are down or latency exceeds limit
- Live Dashboard web status + web trend graph

Technical Overview

A scheduled polling engine is run by the system. Performs all the set probes with raw sockets (ICMP), and TCP handshakes on each poll cycle. The results are stored in the SQLite database, and if a threshold limit is being breached on any one of the metrics then an alert email is sent out. The Flask dashboard web app retrieved the data from SQLite and rendered the graphs from Plotly in the browser. The flask dashboard web app was able to take the data from SQLite and render the plots from Plotly in the browser.

Library	Purpose	Notes

ping3	ICMP ping	Measures RTT; cross-platform
socket	TCP port checks	Built-in; no install needed
APScheduler	Scheduled polling	Periodically polls for checks in background for every N seconds
sqlite3	Data storage	Built-in; lightweight local DB
smtplib	Email alerts	Built-in; connects to Gmail SMTP
Flask	Web dashboard	Lightweight server; serves dashboard UI
Plotly	Charts & graphs	Interactive latency and uptime graphs
requests	HTTP monitoring	Monitor HTTP response codes and response time
threading	Concurrency	Built-in; runs multiple hosts concurrently

Week-by-Week Deliverables

Week 4: Planning & Setup (Current)

- Presentable project plan.
- Design the folder structure of the project in GitHub repository.
- Specify a list of hosts to monitor and the services.
- Conclude tech stack and roles.

Week 5: Core Engine & Alerts

- Using ping3 to implement ping monitor including RTT, packet loss, host-down detection
- Port/service checker using socket module
- Create a alert module: emails using smtplib (thresholds configures by users)
- Setup SQLite database and logging Pipeline
- APScheduler to schedule running polling engine on a periodic basis

Week 6: Dashboard & Integration

- Develop a flask web application and REST endpoints that provide us the metrics using SQLite database
- Generate Plotly graphs: Uptime, packet loss, RTT trend
- Real time link connected with Dashboard has host status overview page.
- All module level integration testing.

Week 7: Testing, Docs & Presentation Prep

- End-to-end tests having several hosts
- Write a README file to get it up and running and instruct the user what to do.
- Record a short demonstration/screenshot for presentation.
- Make and practice concluding slides for live class demo.

Week 8: Final Presentation

- Live demo of our Network Monitoring & Alert System
- Explore the architecture, code walkthrough and lessons learned

Work Division

Team Member	Role	Responsibilities
Sushant Ale Magar	Project Lead & Integration	Overall Architecture, GitHub Management, Connecting all modules together, End-to-End Testing
Bikram Timalisina	Network Monitoring	Ping engine (ping3), TCP port checker (socket), APScheduler setup
Karan Bhatt	Alert System	Email alert module (smtplib), threshold configuration, alert history logging
Rojan Tamang	Dashboard & Visualization	Flask server, Plotly charts, live and historical data views

Learning Expectations

By course end the following course we will learn, and have a practical understanding of:

- Background: how ICMP and TCP function at the inside of an operating system and how to use python to communicate with raw sockets
- Proper structure of a python project, structuring modules, configuration files, handling errors, logging.
- Establishing an alert system to respond to situation automatically, instead of manually checking.

- Develop a basic web dashboard using Flask and dynamically load it with data using Plotly.
- Lastly, working and communicating with others: branching, merging and resolving conflict on the shared codebase on GitHub.
- Finally, collaborating with others by having a shared codebase using GitHub: branching, merging and resolving conflicts.